

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SUBJECT NAME: LANGUAGE TRANSLATORS

SUBJECT CODE: CST54

UNIT- II

Loaders and Linkers: Basic loader functions, machine – dependent and machine – independent loader features. Loader design – Linkage editors, dynamic linking and bootstrap loaders.

2 MARKS

1. Define Loader?

(MAY 2017)

- A loader is a system program that performs the loading function. It brings object program into memory and starts its execution. Many loaders also support relocation and linking.
- Loader is a set of program that loads the machine language translated by the translator into the main memory and makes it ready for execution.

2. Define Linker or Linkage Editors.

(MAY 2017)

Linker or linkage editor to perform the linking operations and a separate loader to handle relocation and loading.

3. What are the two primary tasks of a linker?

(NOV 2013) (MAY 2018)

The two primary tasks of the linker are

1. Relocating relative addresses
2. Resolving external references

4. What are the basic functions of loaders?

(MAY 2016, 2018)

- Loading – brings the object program into memory for execution.
- Relocation – modifies the object program so that it can be loaded at an address different from the location originally specified.
- Linking – combines two or more separate object programs and supplies the information needed to references between them.

5. What are the types of loaders?

The different types of loaders are

1. absolute loader
2. bootstrap loader
3. relocating loader or relative loader
4. linking loader

6. What is an absolute loader? State its disadvantages.

(MAY 2013) (NOV 2017)

- The loader, which is used only for loading, is known as absolute loader.
- The operation of absolute loader is very simple. The object code is loaded to specified locations in the memory. At the end the loader jumps to the specified address to begin execution of the loaded program.
- e.g. Bootstrap loader

Disadvantages:

The disadvantages are,

1. It is need for programmer to specify the actual address at which it will be loaded into memory.
2. It is difficult to use subroutine libraries efficiently.

7. What is the design of absolute loader?

This loader does not perform such functions as linking and program relocation. Its operation is very similar to all functions are accomplished in a single pass.

- The Header record is checked to verify that the correct program has been presented for loading.
- Each Text record is read, the object code contains is moved to the indicated address in memory.
- When the End record is encountered, the loader jumps to the specified address to begin execution of the loaded program.

8. Write the algorithm for absolute loader?

begin

 read Header record

 verify program name and length

 read first text record

while record type ≠ 'E' **do**

begin

 { if object code is in character form, convert into
 internal representation }

 move object code to specified location in memory

 read next object program record

end

 jump to address specified in End record

end

9. What is a bootstrap loader?

(MAY 2012, 2014) (NOV 2014, 2016)

- When a computer is first tuned on or restarted, a special type of absolute loader, called bootstrap loader is executed. This bootstrap loads the first program to be run by the computer – usually an operating system.
- It then jumps to the just loaded program to execute it .The bootstrap itself begins at address 0.
- It loads the OS starting address 0x80. No header record or control information, the object code is consecutive bytes of memory.

10. What is program relocation?

- Program relocation is the method of loading the object code in a different location specified in the object program. Assembler passes this information about the program relocation to the loader through modification record as a part of object program.

- Program relocation is, the execution of the object program using any part of the available and sufficient memory. The object program is loaded into memory wherever there is room for it. The actual starting address of the object program is not known until load time.

11. What is address sensitive program?

(NOV 2011)

- The direct addressing mode is called address sensitive program.
- When operands are specified in memory addressing mode, direct access to main memory, usually to the data segment, is required.
- Direct addressing *Example: LDA ZERO*

12. What are relocating loaders or relative loaders?

Loaders that allow for program relocation are called relocating loaders or relative loaders.

13. List out the functions performed by relocating loaders?

(NOV 2011)

The two functions performed by relocation loaders are

1. Modification record and
 2. Relocation bit.
- In modification record, a modification record M is used in the object program to specify any relocation.
 - In relocation bit, each instruction is associated with one relocation bit and, these relocation bits in a Text record is gathered into bit masks.

14. What is the use of modification record?

Modification record is used for program relocation. Each modification record specifies the starting address and the length of the field whose value is to be altered and also describes the modification to be performed.

15. Define Relocation bit.

- Relocation bit associated with each word of object code. In SIC instructions occupy one word; this means that there is one relocation bit for each possible instruction.
- The relocation bit corresponding to a word of object code is set to 1, the program's starting address is to be added to this word when the program is relocated.
- Bit value 0 indicates no modification is required.

16. What is a bit mask?

(NOV 2014)

- The relocation bits are gathered together following the length indicator in each text record and which is called as bit mask.

- For e.g. the bit mask FFC (111111111100) specifies that the first 10 words of object code are to be modified during relocation.

17. What is the need of ESTAB?

- External Symbol Table (ESTAB) is used to store the name and address of the each external symbol.
- It also indicates in which control section the symbol is defined.

18. What is the use of PROGADDR?

- Program load Address (PROGADDR) is the beginning address in memory where the linked program is to be loaded.
- Its value is supplied to the loader by the operating system.

19. What is the use of CSADDR?

- Control Section Address (CSADDR) contains the starting address assigned to the control section currently being scanned by the loader.
- Its value is added to all relative addresses within the control section to convert them to actual addresses.

20. Write the two passes of a linking loader.

The two passes of linking loaders are

- Pass1: assigns address to all external symbols.
- Pass2: it performs the actual loading, relocation and linking.

21. How are duplicate literal operands handled in machine independent loader? (NOV 2013)

- The duplicate literal used more than once in the program. Only one copy of the specified value needs to be stored.

For how to recognize the duplicate literals

- Compare the character strings defining them. Easier to implement, but has potential problem

E.g., =X'05'

22. Define automatic library search.

In many linking loaders the subroutines called by the program being loaded are automatically fetched from the library, linked with the main program and loaded. This feature is referred to as automatic library search. Linking loaders that support automatic library search must keep track of external symbols that are referred to, but not defined, in the primary input to the loader.

23. List the loader options INCLUDE &DELETE?

The general format of INCLUDE is

INCLUDE program_name (library name)

- This command directs the loader to read the designated object program from a library and treat it as the primary loader input.

The general format of DELETE command is

DELETE csect-name

- It instructs the loader to delete the named control sections from the sets of programs loaded.

24. What are the primary loader commands?

The primary loader input with the loader commands are

- INCLUDE READ (UTLIB)
- INCLUDE WRITE (UTLIB)
- DELETE RDREC, WRREC
- CHANGE RDREC, READ
- CHANGE WRREC, WRITE

25. List the various types of loader design options?

1. Linkage Editors
2. Dynamic linking
3. Bootstrap loaders

26. Give the functions of the linking loader.

The linking loader performs all linking and relocation operations, including automatic library search and loads the linked programs directly loaded into the memory for execution.

27. List the uses of linkage editors?

(MAY 2013) (NOV 2017)

- A linkage editor produces a linked version of the program (often called a **load module** or an **executable image**), which is written to a file or library for later execution.
- It performs relocation of all control sections relative to the start of the linked program.
- Linkage editors can also be used to build packages of subroutines or other control sections that are generally used together.
- Linkage editors often allow the user to specify that external references are not to be resolved by automatic library search – linking will be done later by linking loader – linkage editor + linking loader – savings in space.

28. Why is linking required after a program is translated?

(MAY 2013)

- Linking which combines two or more separate object programs and supplies the information needed to allow references between them

- The routines are automatically retrieved from a library as they are needed during linking.

29. Differentiate linkage editors and linking loaders.

(MAY 2015)

Linking Loader	Linkage Editor
Performs linking and relocation at load time.	Linking is done prior to load time.
Loads the linked program directly into the memory.	Writes a linked version of program, which is later executed by relocating loader.
Linking loader has less flexibility and control.	Linkage editors offer more flexibility and control.

30. Write a note on dynamic linking.

(NOV 2016)

Dynamic linking postpones the linking function until execution time; a subroutine is loaded and is linked to the rest of the program when it is first called(run time).This type of function is usually called **dynamic loading** or **dynamic linking** or **load on call**.

31. Write the advantage of dynamic linking.

- Dynamic linking is used to allow several executing programs to share one copy of a subroutine or library.
- It is often used for references to software object.
- It has the ability to load the routine only when they are needed.
- The dynamic linking avoids the loading of entire library for each execution.

32. Differentiate near and far jump?

- A near jump jumps to a location within the current code segment.
- A far jump is normally used to jump to a location within a different code segment, but it can jump to a location within the current segment as well, if the segment selector in the far address coincides with the value in code segment.

33. List out the features of Machine Independent Loader?

(MAY, NOV 2015)

The Machine independent loader features are

1. Automatic Library Search
2. Loader Options

34. What is the function of MS-DOS linkers?

(NOV 2015)

- MS-DOS LINK is a linker program
- MS-DOS assembler (MASM) produces object modules (.OBJ)

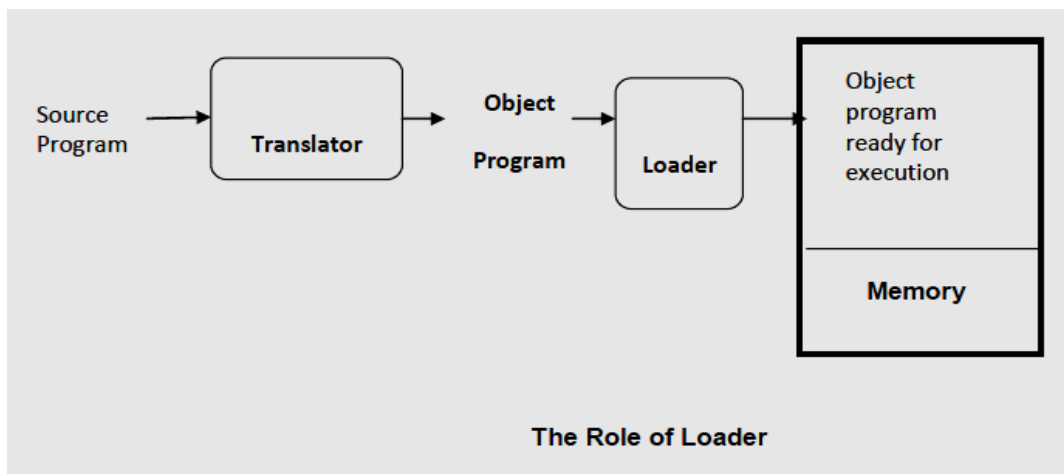
- MS-DOS LINK is a linkage editor that combines one or more modules to produce a complete executable program (.EXE)
- MS-DOS object module
 - ✓ THEADER similar to Header record in SIC/XE
 - ✓ MODEND similar to End record in SIC/XE

11 MARKS

1. Explain with neat block diagram the role of loader and linker. (11 MARKS) (MAY 2013)

Role of Loader

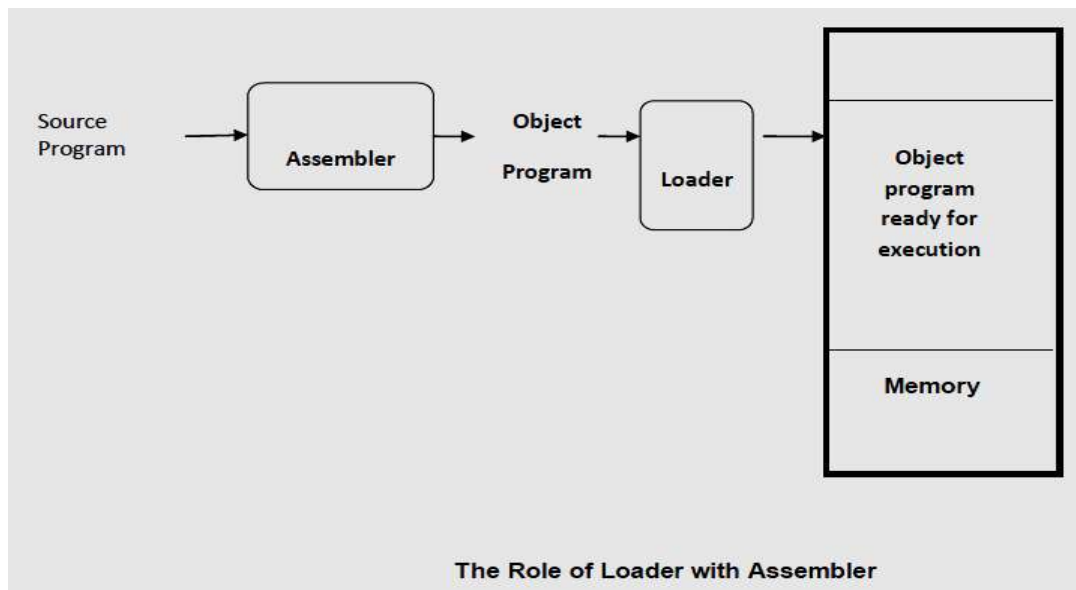
- A *loader* is a system program that performs the loading function. It brings object program into memory and starts its execution. Many loaders also support relocation and linking.
- Loader is a set of program that loads the machine language translated by the translator into the main memory and makes it ready for execution.
- An object program contains translated instructions and data values from the source program and specifies addresses in memory where these items to be loaded.
- The loader is a program which accepts the object program checks, prepares these programs for execution by the computer and initiates the execution.



The loader must perform the following functions:

1. **Loading:** This brings the object program into memory for execution.
 2. **Relocation:** This modifies the object program so that it can be loaded at an address from the location originally specified.
 3. **Linking:** This combines two or more separate object programs and supplies the information needed to allow references between them.
- A loader is a system program that performs the loading function. Many loaders also support relocation and linking. Some systems have a **linker or linkage editor** to perform the linking and a separate loader to handle relocation and loading.
 - In most cases all the program translators (*assemblers and compilers*) on a particular system produce object programs.

Role of loader with assembler



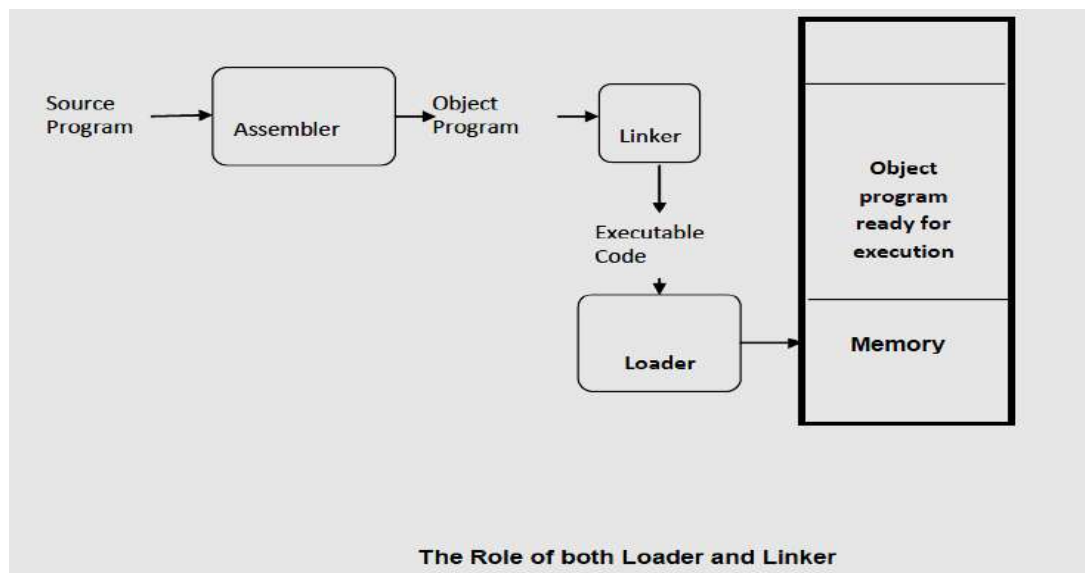
Type of Loaders

The different types of loaders are,

1. absolute loader
2. bootstrap loader
3. relocating loader or relative loader
4. linking - loader

Role of loader and linker

- Linker combines two or more separate object programs and supplies the information needed to allow references between them
- Linker or linkage editor to perform the linking operations and a separate loader to handle relocation and loading.



2. Discuss about Basic Loader Functions. (MAY 2012, 2016, 2017) (OR) Discuss the functions and design of an absolute loader. (NOV 2011, 2015, 2016)

Basic loader functions

The most fundamental function of the loader is bringing the object program into memory and starts its execution.

1. Design of an absolute Loader
2. A simple Bootstrap Loader

Design of an Absolute Loader

This loader does not perform such functions as linking and program relocation, its operation is very simple. All functions are accomplished in a single pass.

- The Header record is checked to verify that the correct program has been presented for loading.
- As each Text record is read, the object code contains is moved to the indicated address in memory.
- When the End record is encountered, the loader jumps to the specified address to begin execution of the loaded program.

Algorithm for an Absolute Loader

begin

 read Header record

 verify program name and length

 read first text record

while record type ≠ 'E' **do**

begin

 { if object code is in character form, convert into
 internal representation }

 move object code to specified location in memory

 read next object program record

end

 jump to address specified in End record

end

Representation of object program

```

HCOPY 00100000107A
T0010001E1410334820390010362810303010154820613C100300102A0C103900102D
T00101E150C10364820610810334C0000454F46000003000000
T0020391E041030001030E0205D30203FD8205D2810303020575490392C205E38203F
T0020571C1010364C0000F1001000041030E02079302064509039DC20792C1036
T002073073820644C000005
E001000

```

(a) Object program

The contents of memory locations for which there is no text record as shown as **xxxx**

Memory address	Contents			
0000	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
0010	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮
0FF0	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
1000	14103348	20390010	36281030	30101548
1010	20613C10	0300102A	0C103900	102D0C10
1020	36482061	0810334C	0000454F	46000003
1030	000000xx	xxxxxxxx	xxxxxxxx	xxxxxxxx ← COPY
⋮	⋮	⋮	⋮	⋮
2030	xxxxxxxx	xxxxxxxx	xx041030	001030E0
2040	205D3020	3FD8205D	28103030	20575490
2050	392C205E	38203F10	10364C00	00F10010
2060	00041030	E0207930	20645090	39DC2079
2070	2C103638	20644C00	0005xxxx	xxxxxxxx
2080	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮

(b) Program loaded in memory

- In the object program, each byte of assembled code is given using its hexadecimal representation in character form.
- The operation code for an **STL** instruction would be represented by the pair of characters “1” and “4”.
- When these are read by the loader, it will occupy two bytes of memory.
- In the instruction as loaded for execution, however, this operation code must be stored in a single byte with hexadecimal value 14.
- Thus each pair of bytes from the object program record must be packed together into one byte during loading.

Disadvantages of Absolute Loader

- The program must specify to the assembler the address in memory where the program is to be loaded.
- If there are multiple subroutines, the programmer must remember the address of each and use that absolute address explicitly in his other subroutines to perform subroutine linkage.
- The programmer must be careful not to assign two subroutines to the same or overlapping.

3. Define bootstrap loader and state the algorithm? (NOV 2011, 2014, 2017) (MAY 2012, 2013, 2018)

- When a computer is first tuned on or restarted, a special type of absolute loader, called *bootstrap loader* is executed. This bootstrap loads the first program to be run by the computer – usually an operating system.
- The bootstrap itself begins at address 0 in the memory of the machine.
- It loads the operating system starting at address 80.
- No header record or control information, the object code is consecutive bytes of memory.

Algorithm for bootstrap loader

The algorithm for the bootstrap loader is as follows

begin

X=0x80 (the address of the next memory location to be loaded)

Loop

A→GETC (and convert it from the ASCII character code to the value of the hexadecimal digit)

save the value in the high-order 4 bits of S

A→GETC

combine the value to form one byte A→ (A+S)

store the value (in A) to the address in register X

X→X+1

End

It uses a subroutine GETC, which is

GETC A→read one character

if A=0x04 then jump to 0x80

if A<48 then GETC

A → A-48 (0x30)

if A<10 then return

A → A-7

return

- Each byte of object code to be loaded is represented on device F1 as two hexadecimal digits.

- However, there is no Header record, End record or control information (such as addresses or lengths).
- The object code from device F1 is always loaded into consecutive bytes of memory, starting at address 80.
- After all of the object code from device F1 has been loaded, the bootstrap jumps to address 80, which begins the execution of the program that was loaded.

4. Explain the Machine Dependent Loader Features? (OR) Write briefly about Relocation technique?

(11 MARKS) (MAY 2014, 2015) (NOV 2017)

The design and implementation of a more complex loader. This loader provides for program relocation and linking. For efficient sharing of the machines, we go for relocatable loader instead of absolute ones. This relocatable loader provides for program relocation and linking.

It contains

- Program Relocation
- Program linking
- Algorithms and data structures

(i) PROGRAM RELOCATION

Loaders that allow for program relocation are called *relocating loaders or relative loaders*.

There are two methods for specifying relocation as part of the object program.

- Using Modification Record.
- Using relocation bit mask.

a) Program relocation using modification record

- A modification record is used to describe each part of the object code that must be changed when the program is relocated.
- Most of the instructions in this program use relative or immediate addressing.
- The only portions of the assembled program that contain actual addresses are the extended format instructions.
- There is one modification record for each value that must be changed during relocation.
- Each modification record specifies the starting address and length of the field whose value is to be altered.
- It then describes the modification to be performed.

Example: **M^ 000007 ^ 05 + COPY.**

- In this example, modification adds the value of the symbol COPY which represents the starting address of the program.
- The modification record scheme is a convenient means for specifying the program relocation.

SIC/XE Relocation Loader algorithm

begin

get PROGADDR from operating system

while not end of input **do**

begin

read next record

while record type ≠ 'E' **do**

begin

read next input record

while record type = 'T' **then**

begin

move object code from record to location ADDR + specified address.

end

while record type = 'M'

add PROGADDR at the location PROGADDR + specified address.

end

end

end

b) Program relocation using Bit Mask

- Relocation bit associated with each word of object code. In SIC instructions occupy one word; this means that there is one relocation bit for each possible instruction.
- The relocation bit corresponding to a word of object code is set to 1, the program's starting address is to be added to this word when the program is relocated.
- Bit value 0 indicates no modification is required.
- The relocation bits are gathered together following the length indicator in each text record and which is called as *bit mask*.

```
HCOPY 00000000107A
T0000001EFFC1400334810390000362800303000154810613C000300002A0C003900002D
T00001E15E000C00364810610800334C0000454F46000003000000
T0010391EFFC040030000030E0105D30103FD8105D2800303010575480392C105E38103F
T0010570A8001000364C0000F1001000
T00106119FE0040030E01079301064508039DC10792C00363810644C000005
E000000
```

If a text record contains fewer than 12 words of object code, the bits corresponding to unused words are set to 0. Thus the bit mask FFC (111111111100) in the first text record specifies that all 10 words of the object code are to be modified during relocation. The mask E00 in the second text record specifies that the first 3 words are to be modified.

SIC Relocation Loader Algorithm

begin

get PROGADDR from operating system

while not end of input **do**

begin

read next record

while record type $\neq$ 'E' **do**

while record type = 'T'

begin

get length = second data

mask bits(M) as third data

for (i=0 ; i< length; i++)

if $M_i = 1$ **then**

add PROGADDR at the location PROGADDR + specified address

else

move object code from record to location PROGADDR + specified address.

read next record

end

end

end

(ii) PROGRAM LINKING

- A program may be composed of many control sections.
- These control sections may be assembled separately.
- These control sections may be loaded at different addresses in memory.
- External references to symbol defined in other control sections can only be resolved after these control sections are loaded into memory.

Example

- Let us consider three separately assembled programs, each of which consists of a single control section.
- Each program contains a list of items (LISTA, LISTB, LISTC).
- The ends of these lists are marked by the labels ENDA, ENDB, ENDC.

The labels on the beginnings and ends of the lists are external symbols. That is they are available for use in linking. Each program contains exactly the same set of references to these external symbols.

External Symbols

- **EXTDEF (external definition)** - The EXTDEF statement in a control section names symbols, called external symbols, that are defined in this (present) control section and may be used by other sections.

ex: EXTDEF LISTA, ENDA
 EXTDEF BUFFER, BUFFEND, LENGTH

- **EXTREF (external reference)** - The EXTREF statement names symbols used in this (present) control section and are defined elsewhere.

ex: EXTREF LISTB, ENDB, LISTC, ENDC
 EXTREF RDREC, WRREC

Example in program linking

Program A

0000	PROGA	START	0	
		EXTDEF	LISTA, ENDA	
		EXTREF	LISTB, ENDB, LISTC, ENDC	

0020	REF1	LDA	LISTA	03201D
0023	REF2	+LDT	LISTB+4	77100004
0027	REF3	LDX	#ENDA-LISTA	050014
0040	LISTA	EQU	*	
0054	ENDA	EQU	*	
0054	REF4	WORD	ENDA-LISTA+LISTC	000014
0057	REF5	WORD	ENDC-LISTC-10	FFFFFF6
005A	REF6	WORD	ENDC-LISTC+LISTA-1	00003F
005D	REF7	WORD	ENDA-LISTA-(ENDB-LISTB)	000014
0060	REF8	WORD	LISTB-LISTA	FFFFC0
		END	REF1	

Program B

0000	PROGB	START	0	
		EXTDEF	LISTB, ENDB	
		EXTREF	LISTA, ENDA, LISTC, ENDC	

0036	REF1	+LDA	LISTA	03100000
003A	REF2	LDT	LISTB+4	772027
003D	REF3	+LDX	#ENDA-LISTA	05100000
0060	LISTB	EQU	*	
0070	ENDB	EQU	*	
0070	REF4	WORD	ENDA-LISTA+LISTC	000000
0073	REF5	WORD	ENDC-LISTC-10	FFFFFF6
0076	REF6	WORD	ENDC-LISTC+LISTA-1	FFFFFFF
0079	REF7	WORD	ENDA-LISTA-(ENDB-LISTB)	FFFFFF0
007C	REF8	WORD	LISTB-LISTA	000060
		END		

Program C

Loc	Source statement			Object code
0000	PROGC	START	0	
		EXTDEF	LISTC, ENDC	
		EXTREF	LISTA, ENDA, LISTB, ENDB	
		.	.	
0018	REF1	+LDA	LISTA	03100000
001C	REF2	+LDT	LISTB+4	77100004
0020	REF3	+LDX	#ENDA-LISTA	05100000
		.	.	
0030	LISTC	EQU	*	
		.	.	
0042	ENDC	EQU	*	
0042	REF4	WORD	ENDA-LISTA+LISTC	000030
0045	REF5	WORD	ENDC-LISTC-10	000008
0048	REF6	WORD	ENDC-LISTC+LISTA-1	000011
004B	REF7	WORD	ENDA-LISTA- (ENDB-LISTB)	000000
004E	REF8	WORD	LISTB-LISTA	000000
		END		

Object Program Example

```

HPROGA 0000000000063
DLISTA 000040END A 000054
RLISTB  ENDB  LISTC  ENDC
:
T0000200A03201D77100004050014
:
T0000540F000014FFFFFF600003F000014FFFFFFC0
M00002405+LISTB
M00005406+LISTC
M00005706+ENDC
M00005706-LISTC
M00005A06+ENDC
M00005A06-LISTC
M00005A06+PROGA
M00005D06-ENDB
M00005D06+LISTB
M00006006+LISTB
M00006006-PROGA
E000020

```

```

HPROGB 000000000007F
DLISTB 000060ENDB 000070
RLISTA  ENDA  LISTC  ENDC
:
T0000360B0310000077202705100000
:
T0000700F000000FFFFFF6FFFFFFF0000060
M000003705+LISTA
M000003E05+ENDA
M000003E05-LISTA
M000007006+ENDA
M000007006-LISTA
M000007006+LISTC
M000007306+ENDC
M000007306-LISTC
M000007606+ENDC
M000007606-LISTC
M000007606+LISTA
M000007906+ENDA
M000007906-LISTA
M000007C06+PROGB
M000007C06-LISTA
E

```

```

HPRGCG 0000000000051
DLISTC 000030ENDC 000042
RLISTA ENDA LISTB ENDB
:
:
T0000180C03100000077100000405100000
:
:
T00000420F0000030000000800000110000000000000
M000001905+LISTA
M000001D05+LISTB
M000002105+ENDA
M000002105-LISTA
M000004206+ENDA
M000004206-LISTA
M000004206+PRGCG
M000004806+LISTA
M000004B06+ENDA
M000004B06-LISTA
M000004B06-ENDB
M000004B06+LISTB
M000004E06+LISTB
M000004E06-LISTA
E

```

- Notice that program A defines LISTA and ENDA, program B defines LISTB and ENDB, and program defines LISTC and ENDC.
- Notice that the definitions of REF1, REF2, ..., to REF7 in all of these three control sections are the same.
- Therefore, after these three control sections are loaded, no matter where they are loaded, the values of REF1 to REF7 in all of these programs should be the same.

REF 1 LISTA

- Program A
 - LISTA is defined in its own program and its address is immediately available. Therefore, we can simply use program counter-relative addressing
- Program B
 - Because LISTA is an external reference, its address is not available now. Therefore an extended-format instruction with address field set to 00000 is used. A modification record is inserted into the object code so that once LISTA's address is known, it can be added to this field.
- Program C
 - The same as that processed in Program B.

REF 2 LISTB + 4

- Program A
 - Because LISTB is an external reference, its address is not available now. Therefore an extended-format instruction with address field set to 00004 is used. A modification record is inserted into the object code so that once LISTB's address is available; it can be added to this field.

- Program B
 - LISTB is defined in its own program and its address is immediately available. Therefore, we can simply use program counter-relative addressing
- Program C
 - The same as that processed in Program A.

REF 3 # ENDA - LISTA

- Program A
 - The difference between ENDA and LISTA (14) is immediately available during assembly.
- Program B
 - Because the values of ENDA and LISTA are unknown during assembly, we need to use an extended-format instruction with its address field set to 0.
 - Two modification records are inserted to the object program – one for +END A and the other for -LISTA.
- Program C
 - The same as that processed in Program B.

REF4 ENDA - LISTA + LISTC

- Program A
 - The difference between ENDA and LISTA can be known now. Only the value of LISTC is unknown. Therefore, an initial value of 000014 is stored with one modification record for LISTC.
- Program B
 - Because none of ENDA, LISTA, and LISTC's values can be known now, an initial value of 000000 is stored with three modification records for all of them.
- Program C
 - The value of LISTC is known now. However, the values for ENDA and LISTA are unknown. An initial value of 000030 is stored with two modification records for ENDA and LISTA.

After Loading into Memory

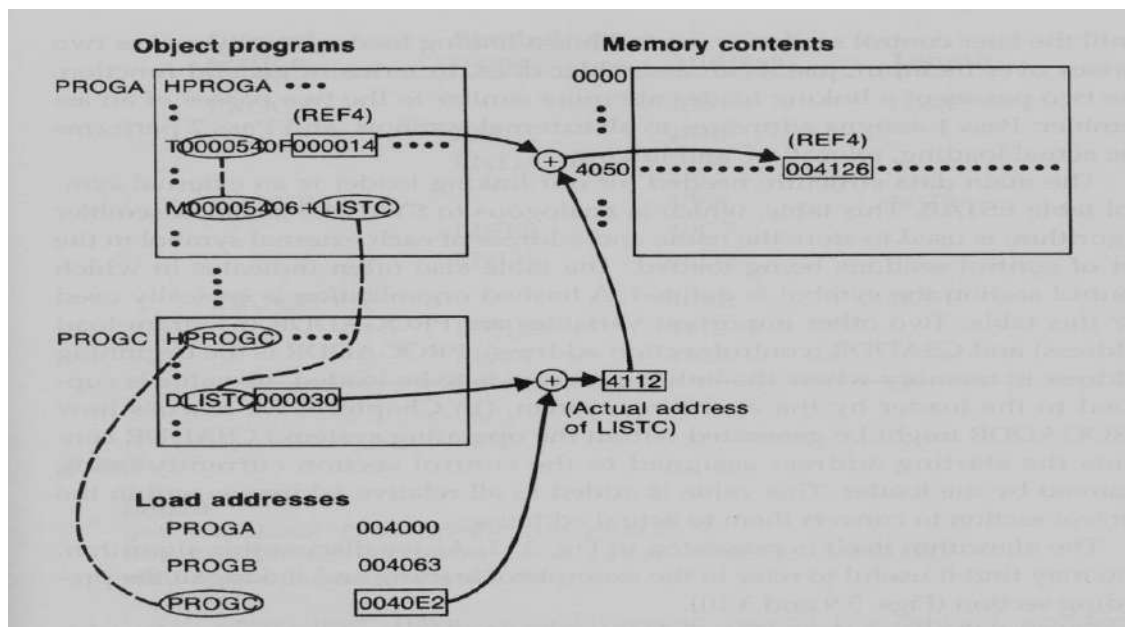
- Suppose that program A is loaded at 004000, program B at 004063, and program C at 0040E2.
- The REF4, REF5, REF6, and REF7 in all of these three programs have the same values.

Memory address	Contents			
0000	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮
3FF0	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
4000				
4010				
4020	03201D77	1040C705	0014....	 ← PROGA
4030				
4040				
4050		00412600	00080040	51000004
4060	000083..			
4070				
4080				
4090			031040	40772027 ← PROGB
40A0	05100014			
40B0				
40C0				
40D0	00	41260000	08004051	00000400
40E0	0083....			
40F0			0310	40407710 ← PROGC
4100	40C70510	0014....		
4110				
4120		00412600	00080040	51000004
4130	000083xx			
4140	xxxxxxxx	xxxxxxxx	xxxxxxxx	xxxxxxxx
⋮	⋮	⋮	⋮	⋮

REF4 AFTER LINKING

▪ In Program A

- The address of REF4 is 4054 (4000 + 54) because program A is loaded at 4000 and the relative address of REF4 within program A is 54.
- The value of REF4 is 004126 because
 - The address of LISTC is 0040E2 (the loaded address of program C) + 000030 (the relative address of LISTC in program C)
 - 0040E2 + 000014 (constant already calculated) = 004126.



▪ In Program B

- The address of REF4 is 40D3 (4063 + 70) because program B is loaded at 4063 and the relative address of REF4 within program A is 70.
- The value of REF4 is 004126 because
 - The address of LISTC is 004112
 - The address of ENDA is 004054
 - The address of LISTA is 004040
 - $004054 + 004112 - 004040 = 004126$

(iii) ALGORITHM AND DATASTRUCTURES OF A LINKING LOADER

Implementation of Loader and Linker

A linking loader makes two passes over its input

- In pass 1: assign addresses to external references
- In pass 2: perform the actually loading, relocation, and linking

Data Structures

- **External symbol table (ESTAB)**
 - Like SYMTAB, store the name and address of each external symbol in the set of control sections being loaded.
 - It needs to indicate in which control section the symbol is defined.
- **Program load Address (PROGADDR)**
 - The beginning address in memory where the linked program is to be loaded.
 - Its value is supplied to the loader by the operating system.
- **Control Section Address (CSADDR)**
 - It contains the starting address assigned to the control section currently being scanned by the loader.
 - This value is added to all relative addresses within the control sections.

Algorithm – PASS1

- During pass 1, the loaders are concerned only with HEADER and DEFINE record types in the control sections to build ESTAB.
- PROGADDR is obtained from OS.
- This becomes the starting address (CSADDR) for the first control section.
- The control section name from the header record is entered into ESTAB, with value given by CSADDR.
- All external symbols appearing in the DEFINE records for the current control section are also entered into ESTAB.
- Their addresses are obtained by adding the value (offset) specified in the DEFINE to CSADDR.

- At the end, ESTAB contains all external symbols defined in the set of control sections together with the addresses assigned to each.
- A Load Map can be generated to show these symbols and their addresses.

A Load Map

Control section	Symbol name	Address	Length
PROGA		4000	0063
	LISTA	4040	
	ENDA	4054	
PROGB		4063	007F
	LISTB	40C3	
	ENDB	40D3	
PROGC		40E2	0051
	LISTC	4112	
	ENDC	4124	

Pass 1:

```

begin
get PROGADDR from operating system
set CSADDR to PROGADDR {for first control section}
while not end of input do
  begin
    read next input record {Header record for control section}
    set CSLTH to control section length
    search ESTAB for control section name
    if found then
      set error flag {duplicate external symbol}
    else
      enter control section name into ESTAB with value CSADDR
    while record type ≠ 'E' do
      begin
        read next input record
        if record type = 'D' then
          for each symbol in the record do
            begin
              search ESTAB for symbol name
              if found then
                set error flag {duplicate external symbol}
              else
                enter symbol into ESTAB with value
                  (CSADDR + indicated address)
            end {for}
          end {while ≠ 'E'}
          add CSLTH to CSADDR {starting address for next control section}
        end {while not EOF}
      end {Pass 1}
    end
  end
end

```

Algorithm - PASS 2

- During pass 2, the loader performs the actual loading, relocation, and linking.
- CSADDR is used in the same way as it was used in pass 1
 - It always contains the actual starting address of the control section being loaded.
- As each text record is read, the object code is moved to the specified address (plus CSADDR).
- When a modification record is encountered, the symbol whose value is to be used for modification is looked up in ESTAB.
- This value is then added to or subtracted from the indicated location in memory.

Pass 2:

```

begin
  set CSADDR to PROGADDR
  set EXECADDR to PROGADDR
  while not end of input do
    begin
      read next input record {Header record}
      set CSLTH to control section length
      while record type ≠ 'E' do
        begin
          read next input record
          if record type = 'T' then
            begin
              {if object code is in character form, convert
               into internal representation}
              move object code from record to location
                (CSADDR + specified address)
            end {if 'T'}
          else if record type = 'M' then
            begin
              search ESTAB for modifying symbol name
              if found then
                add or subtract symbol value at location
                  (CSADDR + specified address)
              else
                set error flag (undefined external symbol)
              end {if 'M'}
            end {while ≠ 'E'}
          if an address is specified {in End record} then
            set EXECADDR to (CSADDR + specified address)
            add CSLTH to CSADDR
          end {while not EOF}
        jump to location given by EXECADDR {to start execution of loaded program}
      end {Pass 2}
    
```

- The linking loader algorithm can be made more efficient if we assign a reference number to each external symbol referred to in a control section.
- This reference number is used (instead of the symbol name) in modification record.
- These simple techniques avoid multiple searches of ESTAB for the same symbol during the loading of a control section.
 - After the first search for a symbol (the REFER records), we put the found entries into an array.
 - Later in the same control section, we can just use the reference number as an index into the array to quickly fetch a symbol's value.

```

HPROGA 000000000063
DLISTA 000040ENDA 000054
02LISTB 03ENDB 04LISTC 05ENDC
:
T0000200A03201D77100004050014
:
T0000540F000014FFFFFF600003F000014FFFFFFC0
M00002405+02
M00005406+04
M00005706+05
M00005706-04
M00005A06+05
M00005A06-04
M00005A06+01
M00005D06-03
M00005D06+02
M00006006+02
M00006006-01
E000020
  
```

5. Explain the machine independent loader features. (11 MARKS) (MAY 2012, 2015)(NOV 2012, 2013)

Machine-independent Loader Features are

- Automatic Library Search
- Loader Options

Automatic Library Search

- The use of automatic library search process for handling external references. This feature allows a programmer to use standard subroutines without explicitly including them in the program to be loaded. The routines are automatically retrieved from a library as they are needed during linking.
- Many linking loaders can automatically incorporate routines from a subprogram library into the program being loaded.
- In most cases there is a standard system library. (E.g., the standard C library)
- Other libraries may be specified by control statements or by parameters to the loader.

- This feature allows the programmer to use subroutines from one or more libraries (mathematical or statistical routines) are a part of the programming language.
- The subroutines called by the program are automatically fetched from the library, linked with the main program, and loaded.
- The programmer does not need to take any action beyond mentioning the subroutine names as external references in the source program. On some systems, this feature is referred to as automatic library call.
- Linking loader that support automatic library search must keep track of external symbols that are referred to, but not defined, in the primary input to the loader.
- At the end of pass 1, the symbols in ESTAB that remain undefined represent unresolved external references.
- The loader searches the library for routines that contain the definitions of these symbols, and processes the subroutines found by this search process exactly as if they had been part of the primary input stream.
- The subroutines fetched from a library in this way may themselves contain external references. It is necessary to repeat the library search process until all references are resolved.
- If unresolved references remain after the library search is completed, they are treated as errors.
- If a symbol (or a subroutine name) is defined both in the source program and in the library, the one in the source program is used first.
- A programmer can make his own library easily on UNIX by using the “ar” command.

Loader Options

- Many loaders allow the user to specify options that modify the standard processing.
- It include such capabilities as a specifying alternative sources of input, changing or deleting external references and controlling the automatic processing of external references.
- The loader option allows the selection of alternative sources of input.

For example, the command

- **INCLUDE program-name (library name)**
 - might direct the loader to read the designated object program from a library program from a library.
- **DELETE csect-name**
 - might instruct the loader to delete the named control sections from the set of programs being loaded.
- **CHANGE name1, name2**
 - might cause the external symbol name1 to be changed to name2 wherever it appears in the object program.

The primary loader input with the loader commands

LIBRARY UTLIB

INCLUDE READ (UTLIB)

INCLUDE WRITE (UTLIB)

DELETE RDREC, WRREC

CHANGE RDREC, READ

CHANGE WRREC, WRITE

NOCALL STDDEV, PLOT, CORREL

- The commands are, use UTLIB (utility library), include READ and WRITE control sections from the library, delete the control sections RDREC and WRREC from the load, the change command causes all external references to the symbol RDREC to be changed to the symbol READ.
- Similarly a reference to WRREC is changed to WRITE.
- Finally, no call to the functions SQRT, PLOT, if they are used in the program.

6. Explain the different options of loader design. (11 MARKS) (NOV 2012, 2013, 2014, 2015, 2016, 2017) (MAY 2014, 2015, 2016, 2017, 2018)

The different loader design options are

- i. Linkage Editors
- ii. Dynamic Linking
- iii. Bootstrap Loaders

(i) LINKAGE EDITORS

- A linking loader performs all linking and relocation operations including automatic library search and loads the linked program directly into memory for execution.
- A **linkage editor** produces a linked version of the program (often called a **load module** or an **executable image**), which is normally written to a file for later execution.
- When the user is ready to run the linked program, a simple relocating loader can be used to load the program into memory.
- The only object code modification necessary is the addition of an actual address to relative values within the program.
- The linkage editor performs relocation of all control sections relative to the start of the linked program.
- All items that need to be modified at load time have values that are relative to the start of the linked program. This means that the loading can be accomplished in one pass with no external symbol table required.

- If a program is to be executed many times without being reassembled, the use of a linkage editor substantially reduces the overhead required.
- Resolution of external references and library searching are only performed once.
- A linking loader searches libraries and resolves external references every time the program is executed.

Advantages

- Linkage editors can perform many useful functions besides simply preparing an object program for execution.
- For example a program (PLANNER) that uses a large number of subroutines.
- Suppose that one sub-routine (PROJECT) used by the program (PLANNER) is changed to correct an error or to improve efficiency.
- After the new version of PROJECT is assembled or compiled, the linkage editor can be used to replace this subroutine in the linked version of the PLANNER.

The linkage editor commands.

```

INCLUDE    PLANNER (PROGLIB)
DELETE    PROJECT          {DELETE from existing PLANNER}
INCLUDE    PROJECT (NEWLIB) {INCLUDE new version}
REPLACE    PLANNER (PROGLIB)

```

- Linkage editors can be used to build packages of subroutines or other control sections that are generally used together.
- This can be useful when dealing with subroutines libraries that support high level programming languages.

The linkage editor also used to combine the appropriate subroutines into a package with a FORTRAN command sequence like

```

INCLUDE READR (FTNLIB)
INCLUDE WRITER (FTNLIB)
INCLUDE BLOCK (FTNLIB)
INCLUDE DEBLOCK (FTNLIB)
INCLUDE ENCODE (FTNLIB)
INCLUDE DECODE (FTNLIB)
.
.
SAVE FTNIO (SUBLIB)

```

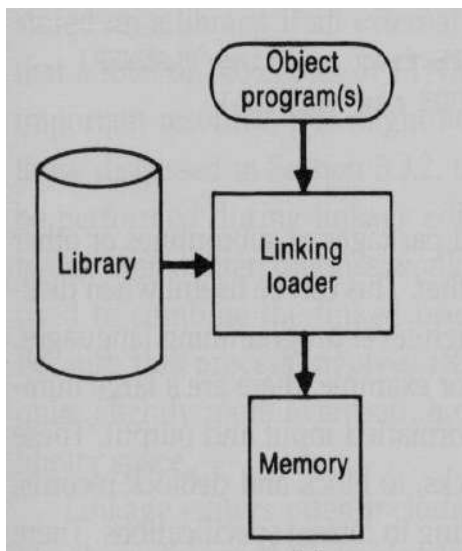
Data blocks
Records
Data items

The difference between a Linkage Editor and a Linking Loader

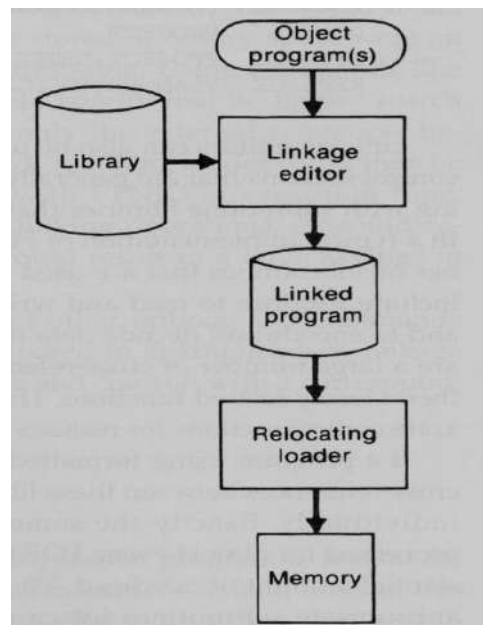
Linking Loader	Linkage Editor
A linking loader performs all linking and relocation operations, including automatic library search, and loads the linked program into memory for execution.	A linkage editor produces a linked version of the program, which is normally written to a file for later execution.
Linkage editors perform linking operations before the program is loaded for execution.	Linking loaders perform these same operations at load time.

Processing of an object program using

a) Linking Loader



b) Linkage Editor



(ii) DYNAMIC LINKING

Dynamic linking postpones the linking function until execution time; a subroutine is loaded and linked to the test of the program when it is first called. This type of function is usually called ***dynamic linking, dynamic loading or load on call.***

Dynamic linking applications

- Dynamic linking is often used to allow several executing programs to share one copy of a subroutine or library.
- For example, run-time support routines for a high level language like C could be stored in ***a dynamic link library.***
- A single copy of the routines in this library could be loaded into memory.
- In an object-oriented system, dynamic linking is often used for references to software objects.
- This allows the implementation of the object and its method to be determined at the time the program is run. (e.g., C++)
- This implementation can be changed at any time, without affecting the program that makes use of the object.
- Dynamic linking also makes it possible for one object to be shared by several programs.

Dynamic Linking Advantage

- The program contains subroutines that correct or diagnose errors in the input data during execution.
- However, the program were completely linked before execution, these subroutines must be loaded and linked every time the program is run.
- Dynamic linking provides the ability to load the routines only when they are needed. If the subroutines involved large or many external references, it can save both time and memory space.

Dynamic Linking Implementation

- The number of different mechanisms that can be used to accomplish the actual loading and linking called subroutine.
- A subroutine that is to be dynamically loaded must be called via an operating system service request.
- This method can also be thought of as a request to a part of the loader that is kept in memory during execution of the program.

(a). Instead of executing a JSUB instruction to an external symbol, the program makes a load-and-call service request to the OS. The parameter of this request is the symbolic name of the routine to be called.

(b). The OS examines its internal tables to determine whether the subroutine is already loaded. If needed, the subroutine from the specified user or system libraries.

(c). Control is passed from the OS to the subroutine being called.

(d). When the called subroutine completes its processing, it returns to its caller (operating system). The OS then returns control to the program that issues the request.

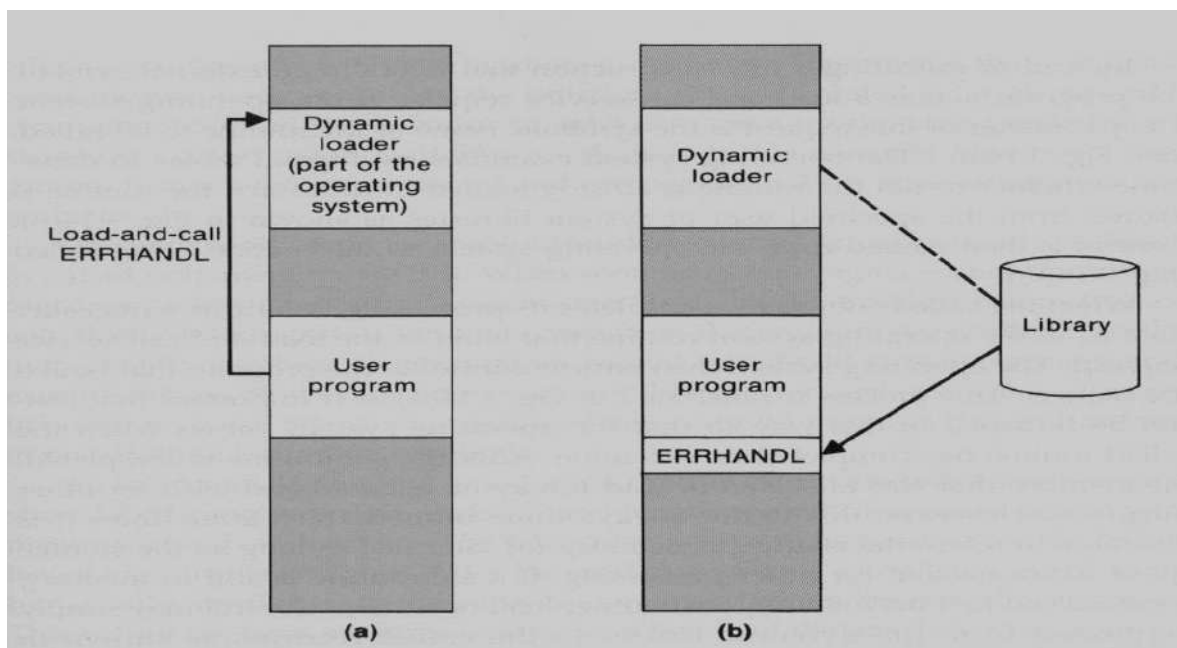
- After the subroutine is completed, the memory that was allocated to it may be released.
- However, often this is not always done immediately. If the subroutine is retained in memory, it can be used by later calls to the same subroutine without loading the same subroutine multiple times.

(e). Control can simply be passed from the dynamic loader to the called routine directly.

Implementation Example

**Issue a Load and Call
service request**

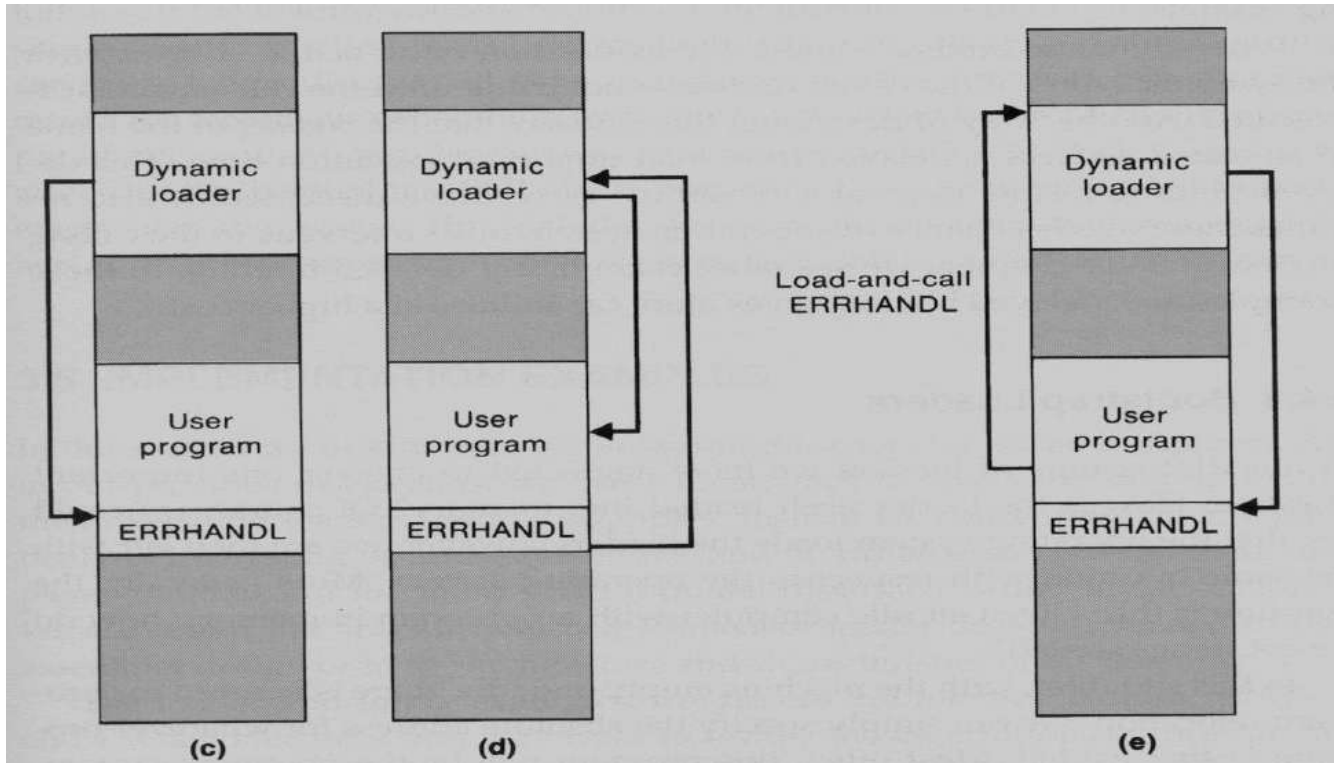
**Load the called subroutine
into memory**



Control is passed
to the loaded
Subroutine

Control is returned to
the loader and later
returned to the user

The called subroutine
this time is already loaded
program



(iii) Bootstrap loaders

- The operating system loads the loader. When computer is started – with no program in memory, a program present in ROM (absolute address) can be made executed – may be OS itself or a Bootstrap loader, which in turn loads OS and prepares it for execution.
- The first record (or records) is generally referred to as a **bootstrap loader** – makes the OS to be loaded.
- Such a loader is added to the beginning of all object programs that are to be loaded into an empty and idle system.

7. Write short notes on Sun OS Linkers? (5 MARKS)

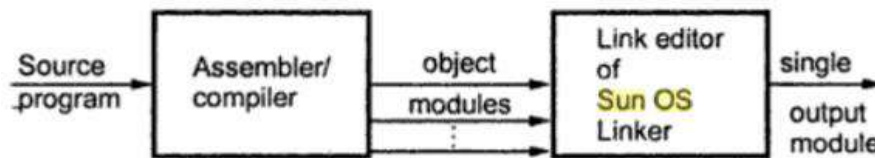
(NOV 2015)

The SUNOS is a UNIX and it is used for scalable processor architecture (SPARC) systems. There are two types of Sun OS linkers.

1. Link-Editor

2. Run-time Linkers

When the program is compiled then at compile time the link editor is used and run-time is used during execution of the program. Fig depicts the role of SunOS Linker.



Link editor of Sun OS linker

Link-Editor:

The link editor program of Sun OS takes many object modules as input. These object modules are obtained after assembling or compiling the source program. The link editor then produces a single output module.

Run-Time Linker:

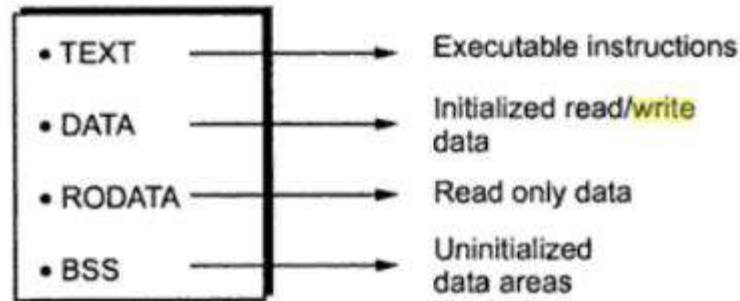
- Uses dynamic linking approach
- Run-time linker binds dynamic executable and shared objects at execution time
- Performs relocation and linking operations to prepare the program for execution

Types of Object Module:

The output module produced by the link editor can be of the following form.

1. Relocatable object module
 2. Static executable
 3. Dynamic executable
 4. Shared object
- A **relocatable object module** –used for further link-editing.
 - A **static executable** – have all symbolic references bound and it is ready to run.
 - A **dynamic executable** – all the symbolic references may need to be bound at run time.
 - A **shared object** –has to be bound with dynamic executable at run time.

The input object module consists of sections. Each section is made up of operations and data. The section has two types of attributes “executable” and “variable”.



An object module contains one or more sections – representing instructions and data area from the source program, relocation and linking information, external symbol table.

SunOS Linkers

- SunOS Linkers are developed for SPARC systems.
- SunOS provides two different linkers – link-editor and run-time linker.
- Link-editor is invoked in the process of assembling or compiling a program – produces a single output module – one of the following types
 - A relocatable object module – suitable for further link-editing.
 - A static executable – with all symbolic references bound and ready to run.
 - A dynamic executable – in which some symbolic references may need to be bound at run time.
 - A shared object – which provides services that can be, bound at run time to one or more dynamic executable.
- An object module contains one or more sections – representing instructions and data area from the source program, relocation and linking information, external symbol table.
- Run-time linker uses dynamic linking approach. Run-time linker binds dynamic executable and shared objects at execution time. Performs relocation and linking operations to prepare the program for execution.

PONDICHERRY UNIVERSITY QUESTIONS

2 MARKS

1. What is address sensitive program? (NOV 2011) (Ref.Qn.No.11, Pg.no.4)
2. List out the functions performed by relocating loaders? (NOV 2011) (Ref.Qn.No.13, Pg.no.4)
3. What is Bootstrap Loader? (MAY 2012 , 2014, 2016) (Ref.Qn.No.9, Pg.no.3)
4. What is an absolute loader? State its disadvantages. (MAY 2013) (NOV 2017) (Ref.Qn.No.6, Pg.no.2)
5. What is a bit mask? (NOV 2014) (Ref.Qn.No.16, Pg.no.5)
6. Why is linking required after a program is translated? Define link editor. (MAY 2013) (Ref.Qn.No.27,28 Pg.no.6,7)
7. What are the two primary tasks of a linker? (NOV 2013) (MAY 2018) (Ref.Qn.No.3, Pg.no.2)
8. How are duplicate literal operands handled in machine independent loader? (NOV 2013) (Ref.Qn.No.21, Pg.no.5)
9. Differentiate near and far jump? (NOV 2014) (Ref.Qn.No.32, Pg.no.7)
10. Differentiate linkage editors and linking loaders. (MAY 2015) (Ref.Qn.No.29, Pg.no.7)
11. List out the features of Machine Independent Loader? (MAY, NOV 2015) (Ref.Qn.No.33, Pg.no.7)
12. What is the function of MS-DOS linkers? (NOV 2015) (Ref.Qn.No.34, Pg.no.8)
13. What are the basic functions of loaders? (MAY 2016, 2018) (Ref.Qn.No.4, Pg.no.2)
14. Write a note on dynamic linking. (NOV 2016) (Ref.Qn.No.30, Pg.no.7)
15. Write short notes on Linkage Editors. (MAY 2017) (Ref.Qn.No.2, Pg.no.2)
16. Define Loader and list out its use case. (MAY 2017) (Ref.Qn.No.1, Pg.no.2)
17. List the uses of linkage editors. (NOV 2017) (Ref.Qn.No.27, Pg.no.6)

11 MARKS

NOV 2011 (REGULAR)

1. Discuss the functions and design of an absolute loader. (Ref.Qn.No.2, Pg.no.11)

(OR)

2. (a) What is dynamic linking? Explain. (6) (Ref.Qn.No.6, Pg.no.30)
(b) Write about bootstrap loaders. (5) (Ref.Qn.No.3, Pg.no.13)

MAY 2012 (ARREAR)

1. Discuss about Basic Loader Functions. (Ref.Qn.No.2, Pg.no.11)

(OR)

2. Explain the machine independent loader features. (Ref.Qn.No.5, Pg.no.25)

NOV 2012 (REGULAR)

1. Discuss about Machine independent loader features. (Ref.Qn.No.5, Pg.no.25)

(OR)

2. Explain the different options of loader design. (Ref.Qn.No.6, Pg.no.27)

MAY 2013 (ARREAR)

1. Define bootstrap loader and state the algorithm for the same. (Ref.Qn.No.3, Pg.no.13)

(OR)

2. Explain with neat block diagram the role of loader and linker. (Ref.Qn.No.1, Pg.no.9)

NOV 2013 (REGULAR)

1. Explain the loader design options. (Ref.Qn.No.6, Pg.no.27)

(OR)

2. (a) Discuss the machine independent loader features in detail. (6) (Ref.Qn.No.5, Pg.no.25)
(b) State and describe the significance of dynamic linking. (5) (Ref.Qn.No.6, Pg.no.30)

MAY 2014 (ARREAR)

1. Write briefly about relocation technique. (Ref.Qn.No.4, Pg.no.14)

(OR)

2. Explain the operation of Dynamic Linking. (Ref.Qn.No.6, Pg.no.27)

NOV 2014 (REGULAR)

1. Explain the following terms
(a) Linkage Editors
(b) Dynamic Linking. (Ref.Qn.No.6, Pg.no.27)
(OR)
2. Write and explain the algorithm for simple bootstrap loader with an example. (Ref.Qn.No.3, Pg.no.13)

MAY 2015 (ARREAR)

1. With examples, explain the machine independent features of a loader. (Ref.Qn.No.5, Pg.no.25)
(OR)
2. (a) State and explain the algorithm of a relocatable loader for SIC/XE. (Ref.Qn.No.4, Pg.no.15)
(b) Write notes on the significance of dynamic linker. (Ref.Qn.No.6, Pg.no.30)

NOV 2015 (REGULAR)

1. Design and write an algorithm of an Absolute loader. (Ref.Qn.No.2, Pg.no.11)
(OR)
2. (a). Write short notes on Sun OS Linkers. (5) (Ref.Qn.No.7, Pg.no.33)
(b). Explain the concept of Dynamic Linking. (6) (Ref.Qn.No.6, Pg.no.30)

MAY 2016 (ARREAR)

1. (a) Write the algorithm for Absolute loader. (6) (Ref.Qn.No.2, Pg.no.11)
(b) Brief about the Loader Design Options. (5) (Ref.Qn.No.6, Pg.no.27)
(OR)
2. (a) What is Dynamic Linking? Explain in detail about the same. (Ref.Qn.No.6, Pg.no.30)
(b) Write notes on Linkage editors. (Ref.Qn.No.6, Pg.no.27)

NOV 2016 (REGULAR)

1. Discuss in detail about absolute loader. (Ref.Qn.No.2, Pg.no.11)
(OR)
2. Differentiate linking loader and linkage editor. (Ref.Qn.No.6, Pg.no.27)

MAY 2017 (ARREAR)

1. Explain in detail about design of Loaders. (Ref.Qn.No.2, Pg.no.11)

(OR)

2. Elaborate in detail about dynamic linking and its applications with an example. (Ref.Qn.No.6, Pg.no.29)

NOV 2017 (REGULAR)

1. (a) With source code, explain the working of bootstrap loader. (7) (Ref.Qn.No.3, Pg.no.13)

- (b) Differentiate linkage editor and linkage loader. (4) (Ref.Qn.No.6, Pg.no.29)

(OR)

2. (a) Discuss machine dependent loader features in detail. (7) (Ref.Qn.No.4, Pg.no.14)

- (b) Discuss the advantages of dynamic linking. (4) (Ref.Qn.No.6, Pg.no.)

MAY 2018 (ARREAR)

1. Explain the features of a linker. Explain about the dynamic linker with an example. (Ref.Qn.No.6, Pg.no.30)

(OR)

2. Write down the steps involved in designing a loader. Explain about bootstrap loader. (Ref.Qn.No.3, Pg.no.13)